

Project Report

Parallel Implementation of Biorthogonal Singular Value Decomposition

Instructor : Prof. Niranjana S Ghaisas

CHANDRA PRAKASH MEENA : CO21BTECH11003

Introduction:

Singular Value Decomposition (SVD) is a technique in linear algebra that factorizes a matrix A into three components:

$$A_{m \times n} = U_{m \times n} \Sigma_{n \times n} V^T_{n \times n}$$

where:

- U is an orthogonal matrix containing the left singular vectors.
- Σ is a diagonal matrix with singular values.
- V^T is the transpose of an orthogonal matrix containing the right singular vectors.

SVD is widely used in data compression, noise reduction, signal processing, and solving linear systems. The biorthogonal SVD method uses Givens rotations to make the matrix progressively more diagonal.

Methodology:

The idea of the **biorthogonal SVD** method is:

- Start with Matrix U as a copy of the input matrix A and $V = I_{n \times n}$
- Apply sequences of Givens rotations (which are simple orthogonal transformations) to gradually orthogonalize the matrix U
- Accumulate rotations into matrices U and V.
- Finally extract singular values from the modified U.

Pseudocode for Serial Biorthogonal SVD:

Input: Matrix A (size m x n)

Output: Matrices U, Σ , V such that $A \approx U \Sigma V^T$

1. Initialize $U = A$, $V =$ Identity matrix (n x n)
2. Compute $N2 =$ sum of squares of all elements in U
3. Repeat until convergence: ($S^{1/2} \leq \epsilon^2 N2^2$)
 - a. Set $S = 0$

- b. For each pair of columns (i, j) where $i < j$:
 - i. Compute $a = \|\text{column } i\|^2$, $b = \|\text{column } j\|^2$, $d = \text{dotProd}(\text{column } i, \text{column } j)$
 - ii. Update $S += d^2$
 - iii. If $d \neq 0$:
 - Compute Givens rotation parameters c, s
 - Apply rotation to columns i and j of U
 - Apply same rotation to columns i and j of V
4. Normalize columns of U
5. Set Σ to a diagonal matrix with computed singular values.

Pseudocode for Parallel Biorthogonal SVD:

Input: Matrix A (size $m \times n$)

Output: Matrices U, Σ , V

1. Initialize $U = A$, $V = \text{Identity matrix } (n \times n)$
2. **Parallel reduction: Compute $N^2 = \text{sum of squares of all elements in } U$**
3. Repeat until convergence ($S^{1/2} \leq \epsilon^2 N^2$) :
 - a. Set $S = 0$
 - b. **Parallel for all column pairs (i, j) where $i < j$:**
 - i. **Parallel reduction: Compute a, b, d**
 - ii. Update $S += d^2$
 - iii. If $d \neq 0$:
 - Compute Givens rotation parameters
 - **Parallel apply Givens rotation to U**
 - **Parallel apply Givens rotation to V**
4. **Parallel normalize each column of U**
5. **Parallel set Σ diagonal entries**

Results:

Matrix size = N (NxN)	Serial Version Execution time (Microseconds)	Parallel Version Execution time (Microseconds)	Speedup
100	243978	142727	1.70940327
200	1713830	244263	7.01633076
300	5937145	1358935	4.36896908
400	13489897	2080659	6.48347326
500	25944873	4317675	6.00899164

Table 1: Performance Comparison of Serial and Parallel Biorthogonal SVD for Different Matrix Sizes.

—The table shows execution times and computed speedup of the parallel implementation (32 threads) relative to the serial version.

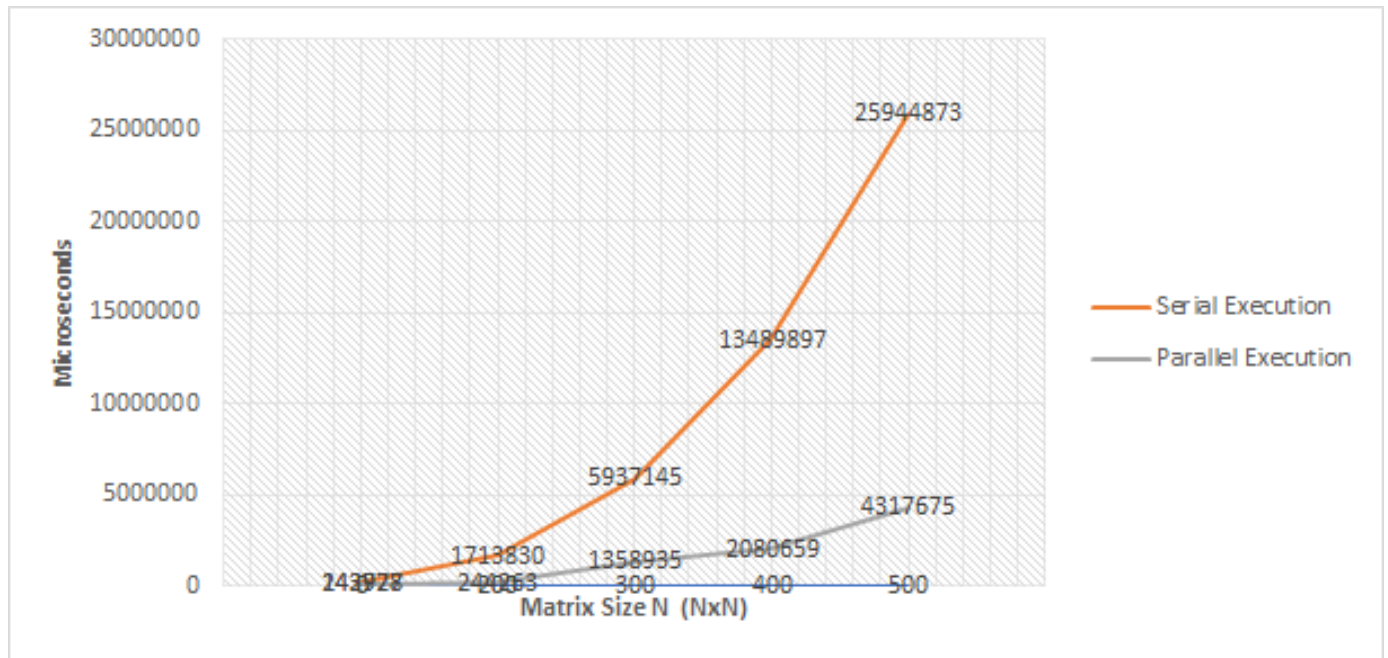


Figure 1: Execution Time Comparison of Serial and Parallel Biorthogonal SVD for Different Matrix Sizes (Threads = 32).

The graph shows how execution time varies with matrix size for both serial and parallel implementations.

Threads	Parallel Version Execution time (Microseconds)
1	6020742
2	4855312
4	3246016
8	2008370
16	1811325
32	1454861
48	1410800

Table 2: Execution Time (in Microseconds) for Parallel Biorthogonal SVD with Varying Number of Threads (Matrix Size = 300×300).

The table illustrates how execution time decreases as the number of threads increases.

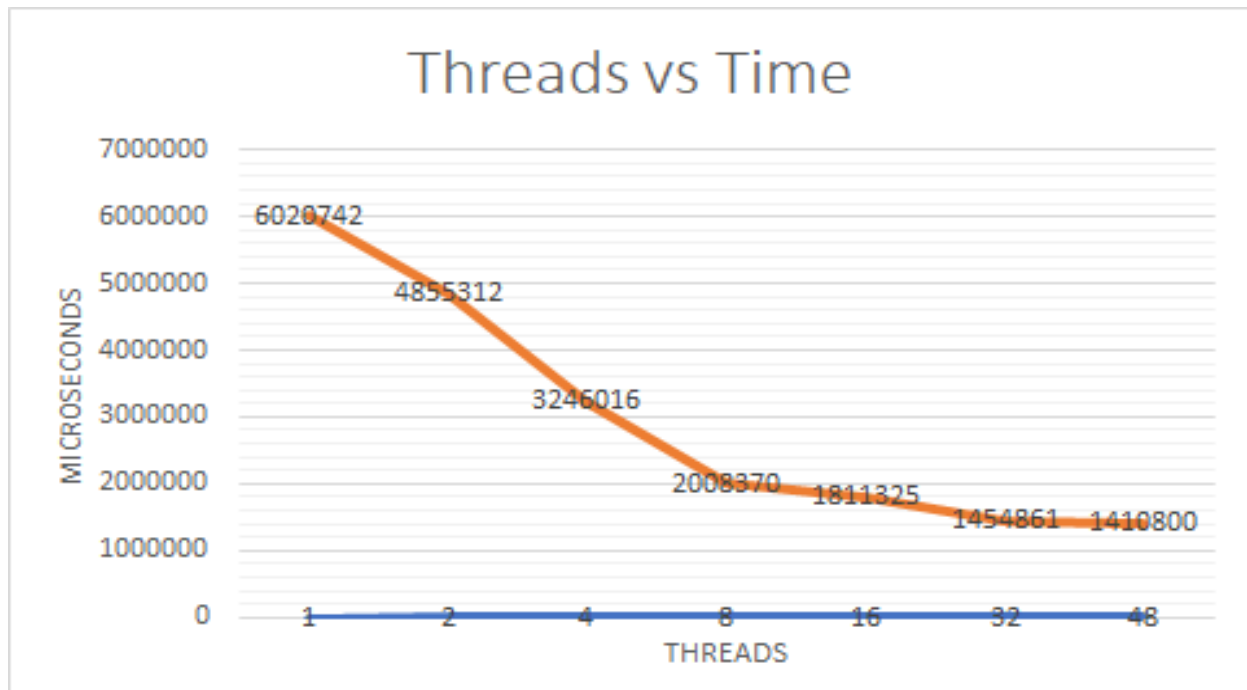


Figure 2: Execution time (in microseconds) vs Number of Threads for Parallel Biorthogonal SVD (Matrix Size = 300×300).

The graph shows a significant decrease in execution time as the number of threads increases, with diminishing returns beyond 32 threads.

Conclusion:

1. We have successfully implemented a **parallelized biorthogonal SVD** using **OpenMP**, achieving significant speedup over the serial version for large matrices.
2. Parallelization was made possible because many steps (rotations, reductions) were independent and suited for multi-threading.

References :

https://www.cs.utexas.edu/~inderjit/public_papers/HLA_SVD.pdf

https://www.cec.uchile.cl/cinetica/pcordero/MC_libros/NumericalRecipesinC.pdf

https://rksmvv.ac.in/wp-content/uploads/2021/04/Gilbert_Strang_Linear_Algebra_and_Its_Application_230928_225121.pdf